Optimal Trade Execution:
From VWAP to the Almgren–Chriss Framework

Lorenzo

github.com/Lorefalo05

Abstract

When a large order is submitted to the market, the price ultimately realized rarely matches the price observed at the moment of decision. This paper develops, from first principles, the quantitative machinery used to minimize that gap: *implementation shortfall* as the canonical measure of execution quality, the empirically-grounded *square-root market impact model*, *VWAP participation scheduling* as a passive benchmark, and the *Almgren–Chriss* mean-variance framework for optimal execution, which admits a closed-form trajectory governed by a single urgency parameter κ. We illustrate the framework with a worked case study of a 50,000-share order, and close with an extension toward *transient impact* (Obizhaeva–Wang), in which market impact decays exponentially rather than being purely temporary or purely permanent.

1 Motivation

Every large order execution faces a trade-off with no free lunch. Trading **fast** consumes liquidity aggressively and incurs steep market impact costs. Trading **slow** reduces impact but leaves the order exposed to adverse price drift before completion — *timing risk*. This paper builds the quantitative tools required to reason about this trade-off, culminating in the Almgren–Chriss (2000) closed-form optimal execution trajectory, with an extension toward more realistic decaying impact dynamics.

2 Problem Setup

Let Q denote the total number of shares to execute (positive for a buy order, negative for a sell), and T the time horizon available for execution. Let $x(t)$ denote the shares remaining at time t, with boundary conditions $x(0) = Q$ and $x(T) = 0$, and define the trading rate as $v(t) = -\dot{x}(t)$.

Let S_0 denote the *decision price* — the price prevailing at the moment the order is initiated — and $\tilde{S}(t)$ the *execution price* realized at time t. The decision price serves as the frictionless benchmark against which all execution costs are measured.

3 Implementation Shortfall

Implementation shortfall (IS) is the standard measure of execution quality: the difference between the planned cost, evaluated at the decision price, and the cost actually incurred.

$$IS = \int_0^T v(t)\,\tilde{S}(t)\,dt \;-\; Q\,S_0 \tag{1}$$

For a buy order ($Q > 0$), a positive IS means more was paid than planned; for a sell order ($Q < 0$), a positive IS means less was received than planned. In either case, positive IS represents a cost of friction in execution.

```
import numpy as np

def implementation_shortfall(decision_price, execution_prices,
                             execution_quantities):
    total_quantity = np.sum(execution_quantities)
    actual_cost = np.sum(execution_prices * execution_quantities)
    planned_cost = total_quantity * decision_price
    return actual_cost - planned_cost
```

4 The Square-Root Market Impact Model

Before optimizing a full execution trajectory, one needs a way to estimate impact ex ante. The empirically robust *square-root law* relates expected impact to order size relative to available liquidity:

$$\text{Impact}\,(\%) = \sigma\sqrt{\frac{Q}{V}} \tag{2}$$

where σ is the daily volatility of the instrument, Q the order size in shares, and V the average daily volume. Converting to dollar terms,

$$\text{Impact Cost} = \text{Impact}\,(\%) \times \text{Price} \times \text{Shares}. \tag{3}$$

```
def estimate_impact_pct(order_size, daily_volume, volatility):
    return volatility * np.sqrt(order_size / daily_volume)

def estimate_impact_dollars(order_size, daily_volume, volatility, price):
    impact_pct = estimate_impact_pct(order_size, daily_volume, volatility)
    return impact_pct * price * order_size
```

Empirically, orders exceeding roughly 5% of daily volume face significant impact; institutional order impact typically falls in the 10–100 basis point range, and the square-root law has been shown to hold robustly across markets and time periods.

5 VWAP: A Benchmark, Not an Optimizer

The *volume-weighted average price* (VWAP) is the average price paid by the typical market participant over a period:

$$\text{VWAP} = \frac{\sum_{i=1}^{n} P_i V_i}{\sum_{i=1}^{n} V_i} \tag{4}$$

Beating VWAP on a buy order means the realized average execution price fell below it. To track VWAP, an order is executed in proportion to expected volume:

$$\text{Shares in period } i = Q \times \frac{V_i^{\text{expected}}}{\sum_{j=1}^{n} V_j^{\text{expected}}} \tag{5}$$

This requires a forecast of the intraday volume curve; volume patterns are, in practice, highly consistent (high at the open, low at midday, high at the close), so historical averages are typically adequate.

```
def calculate_vwap(prices, volumes):
    return np.sum(prices * volumes) / np.sum(volumes)

def vwap_schedule(order_size, expected_volumes):
    expected_volumes = np.asarray(expected_volumes)
    volume_fractions = expected_volumes / np.sum(expected_volumes)
    return order_size * volume_fractions
```

Crucially, VWAP algorithms do not attempt to beat the market — they attempt to match the average. This is precisely the ceiling that motivates a genuine optimizer.

6 Temporary versus Permanent Impact

Price impact from trading decomposes into two distinct components. *Temporary impact* is the displacement caused by immediate demand for liquidity; it reverts once trading stops:

$$\Delta S_{\text{temp}} = \eta\, v \tag{6}$$

where η is the temporary impact coefficient (dollars per share per unit time) and v the trading rate. *Permanent impact* is the lasting price shift caused by information leakage — the market infers the presence of a buyer or seller and re-prices accordingly:

$$\Delta S_{\text{perm}} = \gamma\, n \tag{7}$$

where γ is the permanent impact coefficient (dollars per share) and n the cumulative shares traded.

6.1 The Linear Impact Model

Combining both effects, the execution price when trading at rate v, given n_{prior} shares already executed, is

$$\tilde{S} = S_0 + \gamma\, n_{\text{prior}} + \eta\, v, \tag{8}$$

and the impact cost (the excess over the unaffected price) for n shares is

$$\text{Impact Cost} = n \times (\gamma\, n_{\text{prior}} + \eta\, v)\,. \tag{9}$$

```
def temporary_impact(trading_rate, eta):
    return eta * trading_rate

def permanent_impact(shares_traded, gamma):
    return gamma * shares_traded

def execution_price(base_price, shares_prior, trading_rate, gamma, eta):
    return base_price + eta * trading_rate + gamma * shares_prior
```

6.2 Sliced Execution Cost

When an order is executed in N discrete slices, each slice faces permanent impact compounded from all prior slices:

$$\text{Total Cost} = \sum_i n_i \left(S_0 + \gamma \sum_{j<i} n_j + \eta\, v_i \right) \tag{10}$$

```python
def sliced_execution_cost(base_price, slice_sizes, slice_rates, gamma, eta):
    total_cost = 0.0
    cumulative_shares = 0.0
    for n_i, v_i in zip(slice_sizes, slice_rates):
        exec_price = base_price + gamma * cumulative_shares + eta * v_i
        total_cost += n_i * exec_price
        cumulative_shares += n_i
    return total_cost

def impact_cost(base_price, slice_sizes, slice_rates, gamma, eta):
    total_cost = sliced_execution_cost(base_price, slice_sizes,
                                       slice_rates, gamma, eta)
    baseline_cost = np.sum(slice_sizes) * base_price
    return total_cost - baseline_cost
```

Trading slowly lowers temporary impact but extends exposure to adverse price movement; trading quickly does the reverse. Finding the balance between these two costs is the core problem that the Almgren–Chriss framework solves.

7 The Almgren–Chriss Framework

VWAP disregards three realities: that trading moves the price, that volatility exposes an unfilled order to timing risk, and that traders differ in risk tolerance. Almgren and Chriss (2000) address all three by casting execution as a mean-variance optimization:

$$\min_{\text{trajectory}} \; \mathbb{E}[\text{Cost}] + \lambda \, \text{Var}[\text{Cost}] \tag{11}$$

where $\mathbb{E}[\text{Cost}]$ is the expected impact cost, $\text{Var}[\text{Cost}]$ the variance of cost arising from price volatility, and λ the risk-aversion parameter: higher λ trades a lower variance for a higher expected cost.

7.1 Timing Risk

While shares remain unexecuted, their value is exposed to a random walk $dS = \sigma \, dW$. The variance of total execution cost depends on the entire holding trajectory:

$$\text{Var}[\text{Cost}] = \sigma^2 \int_0^T x(t)^2 \, dt \tag{12}$$

The longer, and the more, shares are held, the greater the exposure to adverse price drift.

7.2 Closed-Form Optimal Trajectory

For the linear impact model above, Almgren–Chriss admits an analytical solution. The optimal shares remaining at time t is

$$x^*(t) = Q \, \frac{\sinh\left(\kappa(T-t)\right)}{\sinh(\kappa T)} \tag{13}$$

where the *urgency parameter* κ is

$$\kappa = \sqrt{\frac{\lambda \sigma^2}{\eta}} \tag{14}$$

κ fuses risk aversion, volatility, and temporary impact into a single number governing trajectory shape. A high κ (high risk aversion or volatility, or low temporary impact) produces a front-loaded, urgent trajectory; a low κ (low risk aversion or volatility, or high temporary impact) produces a slow, even trajectory.

7.3 The Risk-Neutral Limit

As $\lambda \to 0$, $\sinh(\kappa x) \approx \kappa x$, and the trajectory collapses to a straight line:

$$x^*(t) = Q\,\frac{T-t}{T} \tag{15}$$

a uniform trading rate Q/T — equal shares per slice. This is the classical TWAP schedule, recovered here as the zero-risk-aversion special case of Almgren–Chriss.

7.4 Implementation Recipe

1. Compute urgency: $\kappa = \sqrt{\lambda\sigma^2/\eta}$.
2. Generate time points $t_k = k\tau$, $\tau = T/N$, for $k = 0, \dots, N$.
3. Compute the trajectory: if $\kappa > 0$, $x_k = Q\sinh(\kappa(T - t_k))/\sinh(\kappa T)$; if $\kappa = 0$, $x_k = Q(N - k)/N$.
4. Compute the per-slice schedule: $n_k = x_{k-1} - x_k$.

```
def almgren_chriss_trajectory(Q, T, N, lam, sigma, eta):
    tau = T / N
    t_points = np.arange(N + 1) * tau
    kappa = np.sqrt(lam * sigma**2 / eta)

    if kappa > 0:
        trajectory = Q * np.sinh(kappa * (T - t_points)) / np.sinh(kappa * T)
    else:
        trajectory = Q * (T - t_points) / T

    schedule = -np.diff(trajectory)
    return t_points, trajectory, schedule
```

8 Case Study: A 50,000-Share Momentum Order

Consider a momentum signal predicting a +2% move with a 2-hour alpha half-life, requiring the purchase of 50,000 shares of a mid-cap stock within a 4-hour horizon.

8.1 Pre-Trade: Selecting λ from Alpha Decay

Since the signal's edge decays with a 2-hour half-life, urgency should roughly match that decay rate: $\kappa_{\text{target}} = 1/\text{alpha\_halflife}$. Solving $\kappa = \sqrt{\lambda\sigma^2/\eta}$ for λ gives

$$\lambda_{\text{suggested}} = \kappa_{\text{target}}^2\,\frac{\eta}{\sigma^2}, \tag{16}$$

tying risk aversion directly to the expected decay rate of the alpha signal — a fast-decaying signal justifies a front-loaded trajectory even before pure risk aversion is considered.

Parameter	**Value**
Shares (Q)	50,000
Horizon (T)	4 hours
Slices (N)	8 (30-min bars)
Volatility (σ)	1.5%/hour
Permanent impact (γ)	5×10^{-6}
Temporary impact (η)	8×10^{-5}
Decision price	$75.00

Table 1: Case study parameters.

```
Q, T, N = 50_000, 4.0, 8
sigma, gamma, eta = 0.015, 0.000005, 0.00008
decision_price = 75.00

alpha_halflife = 2.0
target_kappa = 1.0 / alpha_halflife
lambda_risk = (target_kappa ** 2) * eta / (sigma ** 2)

t_points, trajectory, schedule = almgren_chriss_trajectory(
    Q, T, N, lambda_risk, sigma, eta
)
```

8.2 Cost Decomposition and Post-Trade Evaluation

Permanent and temporary impact costs, and the timing-risk standard deviation, are computed as

```
tau = T / N
cumulative, perm_cost = 0.0, 0.0
for n_i in schedule:
    perm_cost += gamma * n_i * cumulative
    cumulative += n_i

temp_cost = eta * np.sum(schedule**2 / tau)
expected_cost = perm_cost + temp_cost
std_dev = np.sqrt(sigma**2 * tau * np.sum(trajectory[1:]**2))
```

After simulating fills, the realized implementation shortfall is compared against the pre-trade expected cost via the ratio $IS_{\text{actual}}/\mathbb{E}[\text{Cost}]$: a ratio within 25% of 1 indicates execution performed as expected, a ratio below 0.75 indicates execution beat expectations, and a ratio well above 1 flags a cost overrun worth investigating.

8.3 Simulated Output

Figure 1 shows the full pre-trade / execution / post-trade output for this order, with $\lambda = 0.089$ and the resulting urgency $\kappa = 0.500$.

Four observations follow directly from the plots:

- **The trajectory is front-loaded, as $\kappa = 0.5 > 0$ predicts.** The execution schedule (top right) starts at roughly 11,500 shares in the first slice and tapers to under 3,500 by the last, well above and below the uniform $Q/N = 6{,}250$ benchmark respectively. This reflects the

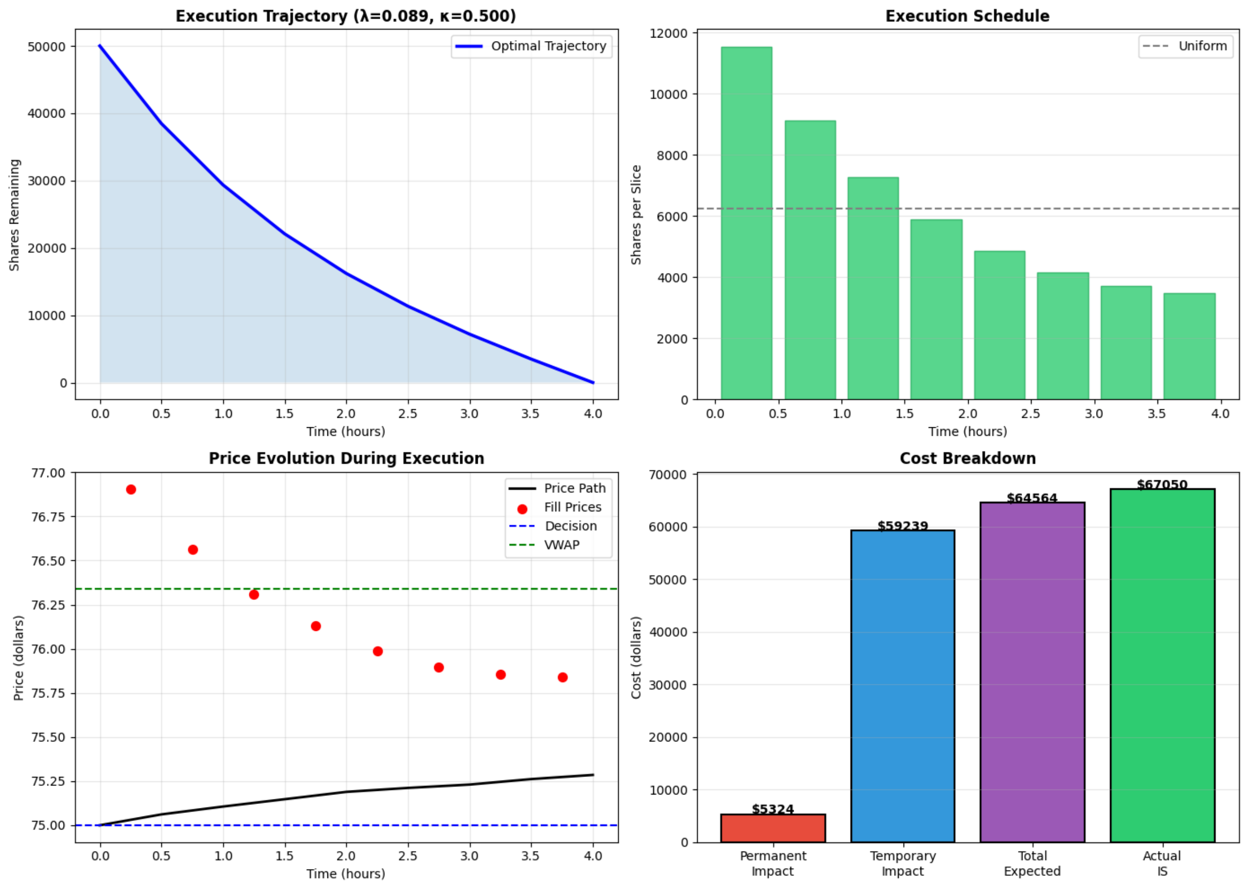

Figure 1: Top left: optimal shares-remaining trajectory, visibly front-loaded. Top right: the corresponding per-slice execution schedule against the uniform (TWAP) benchmark. Bottom left: the simulated price path (black), individual fill prices (red), the decision price (blue, \$75.00), and realized VWAP (green, \$76.34). Bottom right: cost decomposition into permanent impact, temporary impact, total expected cost, and actual implementation shortfall.

pre-trade choice of λ tied to the signal's 2-hour alpha half-life: the model is deliberately urgent because the edge it's trying to capture is decaying.

- **Fill prices fall even though the underlying price path rises.** The black price path (bottom left) drifts up slowly from \$75.00 to about \$75.30 — mostly permanent impact plus noise. Yet the red fill prices *start higher* (near \$76.90) and *decline* toward the price path over the course of execution. This is not a contradiction: the fill price includes the temporary-impact term $\eta \cdot v$, and v is largest in the earliest, most urgent slices. As the trading rate tapers off, the temporary-impact premium shrinks even as the underlying price continues to creep upward, so the two effects move fill prices in opposite directions.

- **Temporary impact dominates the cost, not permanent impact.** The cost breakdown (bottom right) attributes only \$5,324 to permanent impact but \$59,239 — roughly 92% of the \$64,564 total expected cost — to temporary impact. This is the direct consequence of front-loading: an aggressive early trading rate is exactly what the temporary-impact term penalizes most.

- **Realized IS tracks the pre-trade estimate closely.** The actual implementation shortfall ($67,050) sits within about 4% of the $64,564 expected cost — comfortably inside the "good execution" band (within 25% of expectation) defined in the assessment logic of Section 8.

This is the practical payoff of the closed-form solution: the pre-trade cost decomposition (which cost dominates, and why) is available *before* a single share trades, simply from Q, T, σ, γ, η, and the chosen λ.

9 Extension: Transient Impact (Obizhaeva–Wang)

A limitation of the Almgren–Chriss framework is that impact is modeled as either purely temporary (reverting instantly) or purely permanent (never decaying). Real order books sit between these extremes: impact decays gradually as the book refills.

Obizhaeva and Wang (2013) model this with a block-shaped limit order book whose impact decays exponentially at resilience rate ρ:

$$\Delta S(t) = \int_0^t e^{-\rho(t-s)}\, dQ(s) \tag{17}$$

Gatheral and Schied (2011) generalize this to an arbitrary decay kernel G:

$$S(t) = S_0 + \int_0^t G(t-s)\,\dot{x}(s)\,ds \tag{18}$$

These models capture the realistic observation that large trades impact prices for minutes to hours before the order book refills, rather than reverting instantly or never reverting at all.

For the exponential kernel, the impact level updates recursively in $O(N)$:

$$I_t = I_{t-1}\,e^{-\rho\,dt} + \eta\,\mathrm{size}_t \tag{19}$$

```python
def simulate_transient_impact(rho, Q_total, T, N, sigma_daily,
                              eta_impact=0.05):
    dt = T / N
    trade_size = (Q_total / N) * dt
    decay = np.exp(-rho * dt)

    fundamental, impact = 100.0, 0.0
    prices = [fundamental + impact]

    for _ in range(N):
        fundamental += np.random.normal(0, sigma_daily * np.sqrt(dt))
        impact = impact * decay + eta_impact * trade_size
        prices.append(fundamental + impact)

    return np.array(prices)
```

9.1 Simulated Output: Low vs. High Resilience

Figure 2 compares two resilience regimes under identical order flow: a constant trading rate over the horizon $T = 1$ (100,000 shares, $N = 100$ slices, $\sigma_{\text{daily}} = 0.01$), with $\rho = 0.5$ (low resilience, slow decay) against $\rho = 5.0$ (high resilience, fast decay).

Two distinct regimes emerge, and they correspond to the two limiting cases already familiar from Section 6:

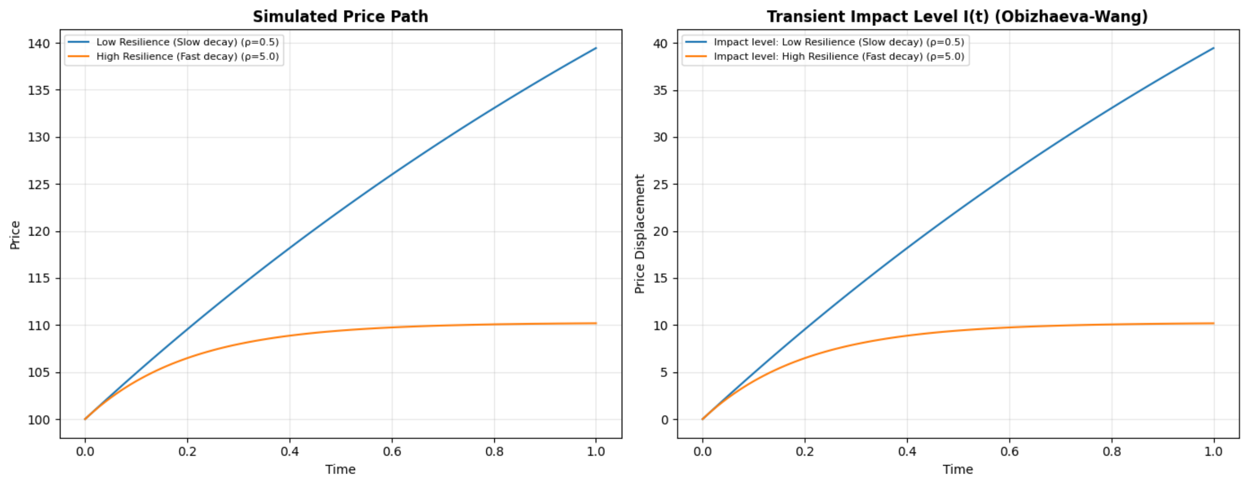

Figure 2: Left: simulated price path under low ($\rho = 0.5$) and high ($\rho = 5.0$) resilience. Right: the corresponding transient impact level $I(t)$ driving the overlay on top of the (unaffected) fundamental price.

- **Low resilience ($\rho = 0.5$, blue curve).** Impact decays slowly relative to the rate of new trades arriving, so each fresh kick $\eta \cdot \text{size}_t$ is added on top of an impact level that has barely decayed since the previous slice. The impact level therefore accumulates almost linearly over the full horizon, reaching roughly 40 in displacement by $t = 1$. This is visually indistinguishable from the *permanent impact* assumption in Almgren–Chriss ($\Delta S_{\text{perm}} = \gamma n$): a slowly-resilient book behaves, over the execution horizon, as if every share traded leaves a permanent mark on the price.
- **High resilience ($\rho = 5.0$, orange curve).** Impact decays quickly enough that within a few slices the loss from decay approximately balances the gain from each new trade, and the impact level converges to a steady-state plateau (around 10 in this simulation) rather than growing without bound. This is the *temporary impact* limit ($\Delta S_{\text{temp}} = \eta v$): the price displacement saturates at a level proportional to the trading rate itself, exactly as the constant-rate temporary-impact term in the linear model would predict.

This is the central insight the transient model adds beyond Almgren–Chriss: *permanent* and *temporary* impact are not two separate physical phenomena to be added together, but the two limiting behaviors of a single decaying-impact process as the resilience parameter $\rho \to 0$ and $\rho \to \infty$, respectively. Real order books sit somewhere in between, and ρ is the parameter that interpolates between them. The key reference for the general theory is Gatheral, Schied, and Slynko (2012), *Transient Linear Price Impact and Fredholm Integral Equations*.

10 Limitations and Future Work

- The linear impact model (η, γ constant) is a simplification — real impact coefficients vary with volatility regime, liquidity conditions, and order-book depth.
- Almgren–Chriss assumes continuous, deterministic trajectories; real execution is discrete and subject to fills, queue position, and adverse selection.
- The transient-impact model here uses a single exponential kernel; the Gatheral–Schied general

kernel $G(t)$ opens the door to power-law decay, consistent with some empirical microstructure studies.

- A natural next step is to recast the transient-impact case as its own optimal-control problem, rather than simulating a fixed-rate schedule, and compare the resulting trajectory to the classical Almgren–Chriss solution.

References

[1] Almgren, R., & Chriss, N. (2000). Optimal Execution of Portfolio Transactions. *Journal of Risk*, 3, 5–40.

[2] Obizhaeva, A., & Wang, J. (2013). Optimal Trading Strategy and Supply/Demand Dynamics. *Journal of Financial Markets*, 16(1), 1–32.

[3] Gatheral, J., & Schied, A. (2011). Optimal Trade Execution under Geometric Brownian Motion in the Almgren and Chriss Framework. *International Journal of Theoretical and Applied Finance*, 14(3), 353–368.

[4] Gatheral, J., Schied, A., & Slynko, A. (2012). Transient Linear Price Impact and Fredholm Integral Equations. *Mathematical Finance*, 22(3), 445–474.

Code and full implementation: github.com/Lorefalo05